

GPU 101

How a machine for pictures learned to compute $A + B$

Minjae Gwon
POSTECH
September 10, 2026

그림을 그리던 기계는 어떻게 A와 B를 더하는 계산기가 됐을까요? 오늘은 이 질문에서 출발해, GPU가 왜 지금과 같은 구조를 갖게 됐는지 이야기해보겠습니다.

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

- Rendering and shaders
- GPGPU and CUDA

3. The execution engine

- Warps and SIMT
- SMs and GPU organization

4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

Contents

발표는 네 부분입니다. GPU를 이해하려는 동기에서 시작해, 그래픽이 범용 계산으로 이어진 과정을 보겠습니다. 이어서 GPU 안의 실행 엔진을 살펴보고, 마지막에는 PyTorch의 덧셈 한 줄이 그 기계에서 처리되는 과정을 따라가겠습니다.

1. Motivation

- Why GPUs matter
- Parallel work in pictures

2. From pictures to programs

3. The execution engine

4. Following A + B

Motivation

먼저 GPU를 쓰면서 마주치는 궁금증부터 짚어보겠습니다.

What should make more sense after this talk?

- **The GPU you observe**

What are VRAM and all those execution resources?

- **The frameworks you use**

What sits below a PyTorch or Hugging Face operation?

- **The platform NVIDIA built**

How did graphics hardware become a general compute platform?

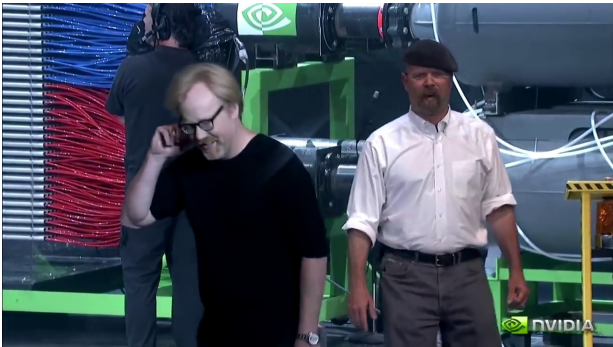
Pictures → Execution engines → $C = A + B$

What should make more sense after this talk?

GPU를 쓰다 보면 nvidia-smi를 켜서 메모리 사용량을 보거나, 모델을 올렸는데 왜 기다리는지 궁금해지는 순간이 있습니다. PyTorch나 Hugging Face에서는 몇 줄로 끝나는 작업이 아래에서는 어떻게 실행되는지도 잘 보이지 않습니다.

오늘은 VRAM에 데이터를 올리는 것과 GPU가 계산하는 것이 어떻게 연결되는지, 프레임워크의 요청 아래에서 무슨 일이 일어나는지 이해할 기본 그림을 만들겠습니다. NVIDIA가 그래픽 장치를 계산 플랫폼으로 확장한 과정도 이 그림의 출발점입니다. 먼저 많은 일을 함께 처리한다는 것이 어떤 모습인지 시연으로 보겠습니다.

One image. Very different ways to paint it.



Parallel work becomes easy to see when the output is a picture.

CPU vs. GPU

One paintball at a time.

Many paintballs together.

▶ [Watch the demo](#)

One image. Very different ways to paint it.

[진행 메모: 시간이 허용되면 사진이나 오른쪽 링크를 눌러 영상을 재생한다. 아래는 읽을 대사.]

MythBusters가 2008년 NVIDIA 행사에서 보여준 시연입니다. 한쪽은 페인트볼을 하나씩 쏘고, 다른 쪽은 많은 페인트볼을 함께 쏘서 그림을 만듭니다. 같은 그림을 완성하는 두 방식의 차이에 주목해보 시면 됩니다.

실제 CPU도 병렬로 계산할 수 있고, 어느 방식이 빠른지는 작업에 따라 달라집니다. 이 시연이 잘 보여주는 것은 여러 위치의 작업을 함께 진행하는 모습입니다. 그림을 그리는 일에는 왜 이런 병렬성이 많았을까요?

NVIDIA NVISION 08 / MythBusters demo · video re-upload: HOSTKEY

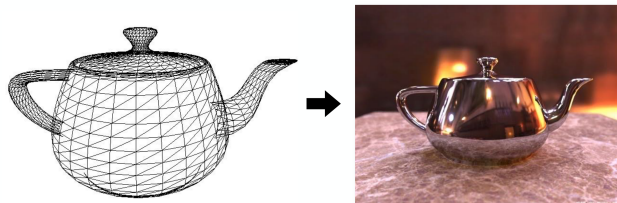
- Rendering and shaders
- GPGPU and CUDA

From pictures to programs

그림 한 장을 만드는 데 어떤 반복 계산이 필요한지 보겠습니다.

The original deadline: draw the next frame

3D rendering



Input: description of a scene:

3D surface geometry (e.g., triangle mesh)
surface materials, lights, camera, etc.

Output: image of the scene

Image credit: Henrik Wann Jensen

Geometry + materials + lights + camera → an image

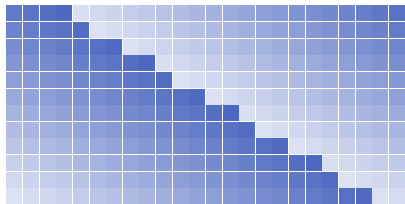
Stanford CS149, Fall 2025 · slide 5 · image: Henrik Wann Jensen

The original deadline: draw the next frame

게임 속에서 햇빛이 비치는 호숫가를 걷는다고 생각해봅시다. 카메라가 움직이면 화면에 보이는 물결과 가려짐, 반사 모습이 달라집니다. 컴퓨터는 물체의 삼각형들, 재질, 조명, 카메라 위치를 가지고 다음 화면을 계산해야 합니다. 이 작업이 렌더링입니다.

화면의 주전자도 마찬가지입니다. 왼쪽에는 표면의 모양을 나타내는 기하 정보가 있고, 오른쪽에는 빛과 재질을 반영한 이미지가 있습니다. GPU는 원래 이런 3차원 그래픽을 빠르게 처리하기 위해 발전했습니다. 게임에서는 이 계산을 반복해 다음 장도 정해진 시간 안에 계속 만들어야 합니다.

Millions of positions. The same kind of calculation.



Different positions, different inputs

 1920×1080

output pixels per frame

 $\times 60 \text{ frames / second}$ $\approx 124 \text{ million}$

output pixels per second

The opportunity is repeated work across independent data.

Millions of positions. The same kind of calculation.

1920 곱하기 1080 화면에는 약 200만 개의 픽셀이 있습니다. 이것을 초당 60장 만들면 매초 약 1억 2천4백만 개의 출력 픽셀을 만들어야 합니다. 이것은 연산 횟수가 아니라 출력 픽셀 수입니다. 하나의 픽셀을 정하기 위해서는 훨씬 많은 계산이 필요할 수 있습니다.

여기서 중요한 것은 단순히 숫자가 크다는 사실만이 아닙니다. 수면의 이 지점에서도 빛의 반사를 계산하고, 바로 옆 지점에서도 비슷한 계산을 합니다. 위치와 방향 같은 입력은 다르지만 적용할 계산법은 같을 수 있습니다. 서로 다른 위치에 대해 비슷한 계산을 아주 많이 한다는 성질이 GPU가 활용할 수 있는 병렬성의 출발점입니다.

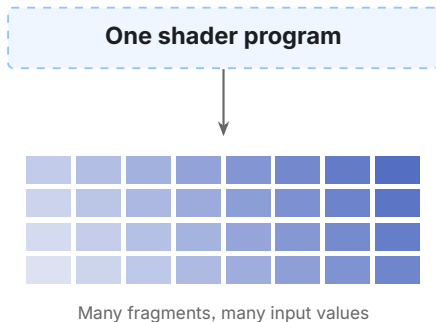
A shader is a program for a graphics stage

ONE FRAGMENT'S WORK

```
base    = read_surface_color(uv)
light   = max(dot(normal, light_dir), 0)
output  = base * light
```

- Programmable behavior replaces a fixed formula.
- A fragment is a candidate contribution to a pixel.

Write how one surface point is shaded; apply it across the surface.



Concept: Stanford CS149, Fall 2025 · slides 11–14 · simplified pseudocode

A shader is a program for a graphics stage

한 표면의 여러 위치에는 같은 계산을 반복하지만, 어떤 계산을 반복할지는 소재에 따라 고르고 싶습니다. 물과 벽돌의 표면색을 계산하는 방식은 다를 테니까요. 정해진 기능을 수행하던 그래픽 파이프라인의 일부를 프로그래머가 작성한 코드로 바꿀 수 있게 된 것이 프로그래머블 셰이더입니다. 셰이더는 그래픽 처리의 한 단계에서 실행되는 프로그램입니다.

삼각형을 화면에 펼치면 화면 위치별로 처리할 조각, 즉 프래그먼트가 생깁니다. 여기에 프래그먼트 셰이더를 적용합니다. 이 조각은 가려짐 검사 등을 거쳐 최종 픽셀에 기여할 수 있는 후보입니다.

화면의 코드는 표면색을 읽고, 빛을 얼마나 정면으로 받는지 구한 다음, 둘을 곱합니다. 같은 표면의 여러 조각에 이 프로그램을 반복 적용합니다. 계산법은 같아도 위치별 입력이 달라 결과색은 달라집니다.

Do those numbers have to be colors?

THE GRAPHICS INTERPRETATION



THE COMPUTE INTERPRETATION



GPGPU: General-Purpose Computing on GPUs

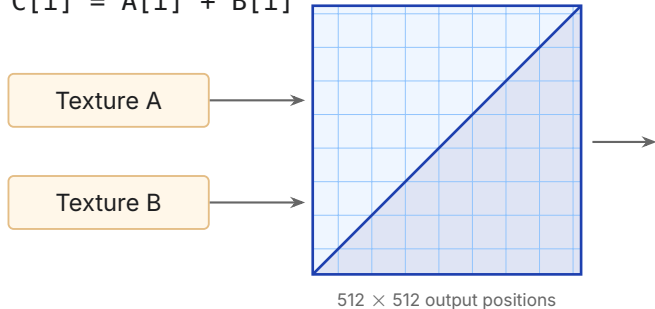
Do those numbers have to be colors?

이 구조를 보던 연구자가 이런 생각을 했다고 해봅시다. 저 숫자가 꼭 색깔이어야 할까? GPU는 입력값을 사람이 물의 색이라고 부르는지, 온도라고 부르는지 신경 쓰지 않습니다. 숫자를 읽고 계산한 다음 결과를 저장할 뿐입니다.

위쪽은 그래픽의 언어입니다. 각 위치의 입력을 셰이더에 넣어 출력색을 만듭니다. 아래쪽에서는 이름만 바꿨습니다. 각 데이터 포인트의 입력을 함수에 넣어 결과값을 만듭니다. 기존 기계를 새로 발명한 것이 아니라, 원래 하던 일에서 그림이라는 의미를 벗겨낸 것입니다. 이런 식으로 많은 데이터에 같은 계산을 병렬로 적용할 수 있다면 그래픽이 아니어도 GPU를 활용할 수 있습니다. 이것이 초기 GPGPU의 핵심 관점입니다.

To add two arrays, first draw two triangles

$$C[i] = A[i] + B[i]$$



- 1 Set the output image size.
- 2 Cover it with two triangles.
- 3 Add values in a fragment shader.

Interpret the output as an array.

A draw call to the GPU; an array operation to the researcher.

To add two arrays, first draw two triangles

발견과 실제 사용 사이에는 재미있는 장애물이 있었습니다. 우리가 하고 싶은 것은 A와 B의 같은 원소를 더해서 C에 쓰는 것뿐인데, GPU에 일을 시키는 통로가 그래픽 API였습니다. 그래서 계산을 그림 그리기로 번역해야 했습니다.

512 곱하기 512개의 원소를 처리하려면 먼저 출력 이미지 크기를 그렇게 설정합니다. 입력 배열은 원래 표면의 무늬를 저장하던 텍스처에 넣습니다. 그다음 출력 영역을 정확히 덮는 삼각형 두 개를 그립니다. 그러면 각 위치에 프래그먼트 셰이더가 실행되고, 셰이더는 색을 만드는 대신 A와 B의 값을 읽어 더합니다. 출력된 이미지를 숫자 배열로 해석하면 계산이 끝납니다. GPU에게는 사각형을 그리는 작업이었지만, 연구자에게는 배열 전체에 함수를 적용하는 작업이었던 것입니다.

“All this just to do A + B”



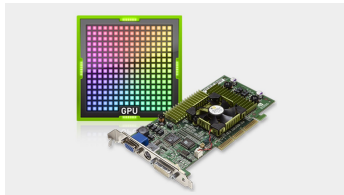
- The entry point still speaks graphics.
- Fragment programs have restricted output locations.
- Sharing data between executions is constrained.

“All this just to do A + B”

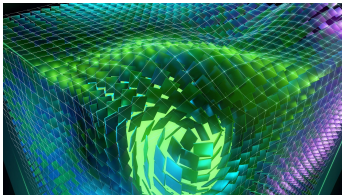
NVIDIA의 2006년 자료도 이 상황을 고작 A 더하기 B를 하려고 이 모든 것을 한다고 표현했습니다. 불편함은 코드가 길어진다는 데서 끝나지 않았습니다.

지금 본 초기 프래그먼트 경로에서는 결과가 그 조각에 대응하는 그래픽 출력 위치로 나갔습니다. 일반 배열의 원하는 원소에 결과를 쓰거나, 다른 실행의 중간값을 주고받는 계산을 표현하는 데 제약이 있었습니다. 입구부터 내부 규칙까지 그림을 그리는 장치에 맞춰져 있었던 것입니다. 그렇다면 이 우회 경로를 정식 계산 인터페이스로 바꿀 수 있지 않을까요?

CUDA made compute a first-class interface

**1999**

GeForce 256

**2006**

CUDA architecture

launch()

CUDA Toolkit 1.0

2007

CUDA Toolkit 1.0

Allocate buffers. Run a kernel. Read and write data directly.

Images / 2006: NVIDIA corporate timeline · Toolkit 1.0: NVIDIA guide, June 2007

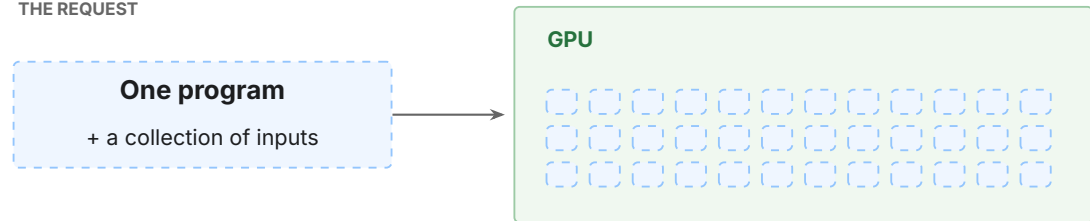
CUDA made compute a first-class interface

이 변화를 연표로 정리해보겠습니다. 왼쪽의 1999년 GeForce 256은 그래픽 GPU의 이정표입니다. 여기서 계산 플랫폼으로 넘어가는 핵심 전환은 가운데와 오른쪽입니다. NVIDIA는 2006년에 CUDA 아키텍처를 공개했고, 2007년에는 CUDA Toolkit 1.0을 출시했습니다.

이제 GPU 메모리에 일반 배열 버퍼를 준비하고, 그 주소를 사용해 원하는 원소를 읽거나 결과를 쓸 수 있습니다. GPU에서 실행할 프로그램을 커널이라고 하고, 그 실행을 요청하는 것을 launch라고 부릅니다. 삼각형을 그리는 절차 없이 계산을 직접 요청하는 것입니다. 정해진 범위에서 실행끼리 데이터를 공유하고 동기화하는 기능도 제공합니다. 계산을 시키는 입구와 가능한 동작이 함께 바뀐 것입니다.

“Run this program many times.”

THE REQUEST



The data-parallel structure survives.

Many logical executions

Now look inside the machine that accepts that request.

“Run this program many times.”

사용자가 이제 하고 싶은 말은 이 프로그램을 많이 실행해달라는 것입니다. 화면을 설정하고 삼각형을 그리라는 말 대신, 실행할 프로그램과 데이터를 직접 건넵니다. 그래픽이라는 외피를 벗겨도 많은 데이터에 같은 계산을 적용한다는 성격은 그대로 남아 있습니다.

여기서 연대기를 마치고 설계 관점으로 바꿔보겠습니다. 이어지는 설명은 실제 발명 순서를 재현하는 것이 아니라, 이런 요청을 처리할 계산기를 만든다면 어떤 구조가 필요할지 생각해보는 과정입니다.

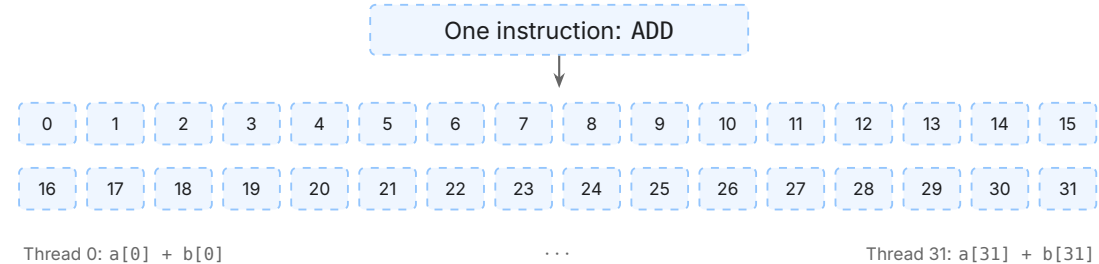
3. The execution engine

- Warps and SIMT
- SMs and GPU organization

The execution engine

이제부터 내부 구조와 프로그래밍 모델은 NVIDIA CUDA GPU를 기준으로 보겠습니다. 수많은 실행에 같은 명령을 어떻게 효율적으로 전달할까요?

Hardware groups 32 logical threads into a warp



SIMT Single Instruction, Multiple Threads

A warp groups thread execution; execution units do the work.

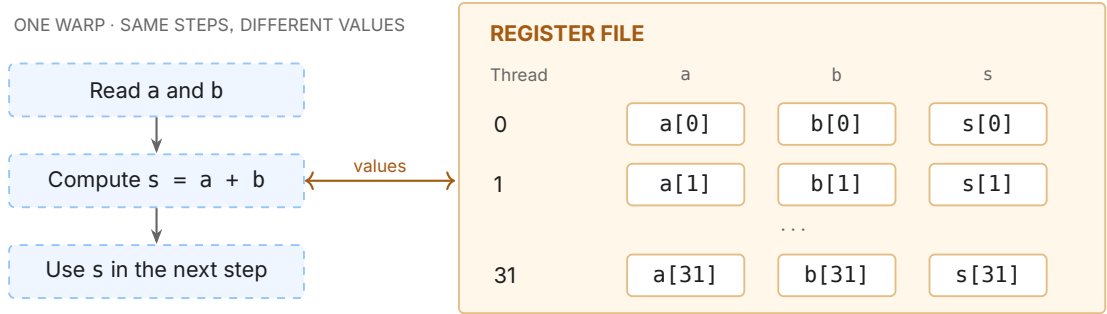
Hardware groups 32 logical threads into a warp

여러 데이터에 대해 덧셈을 해야 한다고 해봅시다. 각 계산의 입력은 다르지만, 지금 해야 할 명령은 모두 덧셈입니다. 하나의 명령으로 여러 데이터의 계산을 함께 진행하면 명령을 전달하고 제어하는 비용을 나눌 수 있습니다.

프로그램의 개별 논리적 실행을 thread라고 합니다. Warp는 하드웨어가 실행을 묶어 관리하는 32개 논리적 thread의 묶음입니다. 여기서 보는 NVIDIA CUDA GPU는 프로그램에서 정하는 더 큰 작업 묶음인 block 안에서, 연속된 번호의 thread를 32개씩 묶습니다. block을 어떻게 구성하는지는 뒤의 코드 예제에서 보겠습니다.

파란 칸은 각 thread의 실행을 나타내며, 특정 연산기와 일대일로 대응하지는 않습니다. 프로그래머는 thread마다 할 일을 쓰고, 하드웨어는 그 thread들의 명령을 묶어 처리합니다. 이것이 SIMT, Single Instruction, Multiple Threads입니다. 조건에 따라 일부 thread만 명령에 참여할 수도 있습니다. 그러면 이 실행들을 진행하려면 하드웨어가 무엇을 기억하고 어떤 일을 해야 할까요?

Each thread needs values that survive an instruction



A register file holds working values; each thread has its own registers.

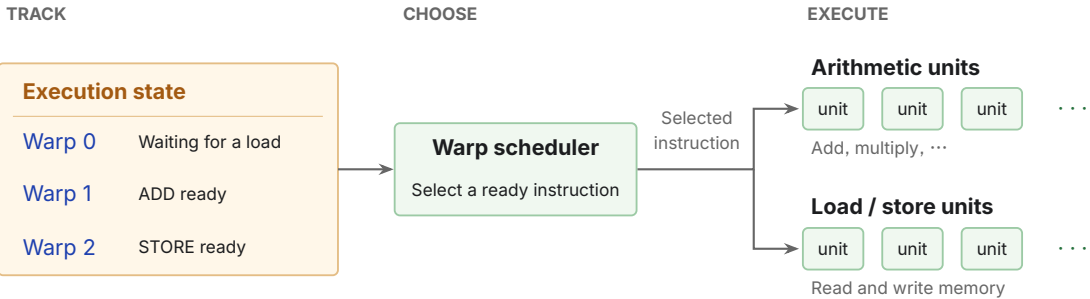
Each thread needs values that survive an instruction

이제 이 thread들의 실행에 필요한 하드웨어를 생각해봅시다. 먼저 덧셈에 쓸 값을 읽었다고 하면, 덧셈 명령이 그 값을 사용할 때까지 어디엔가 보관해야 합니다. 덧셈이 끝나도 다음 계산에서 결과를 다시 쓸 수 있으니, 명령 사이에 값을 남겨둘 자리가 필요합니다.

이런 현재 작업값을 담는 저장 공간이 레지스터입니다. 오른쪽처럼 많은 레지스터를 모아 놓은 하드웨어 저장 구조를 레지스터 파일이라고 부릅니다. 디스크에 저장하는 파일이라는 뜻은 아닙니다. 그림의 가로줄마다 서로 다른 thread의 입력 a, b와 결과 s가 있습니다. 저장 장치는 함께 구성해도, 프로그램에서 각 thread가 사용하는 레지스터 값은 자기 것입니다.

이제 값을 보관할 곳은 생겼습니다. 그런데 여러 warp를 실행하고 있다면, 어떤 실행이 어디까지 왔고 다음에 누구의 명령을 처리해야 하는지도 알아야 하지 않을까요?

Track progress, choose an instruction, execute it



Required roles; unit counts and connections are illustrative.

Track each warp’s state. Issue selected instructions to execution units.

NVIDIA Volta Tuning Guide · Instruction scheduling · conceptual roles

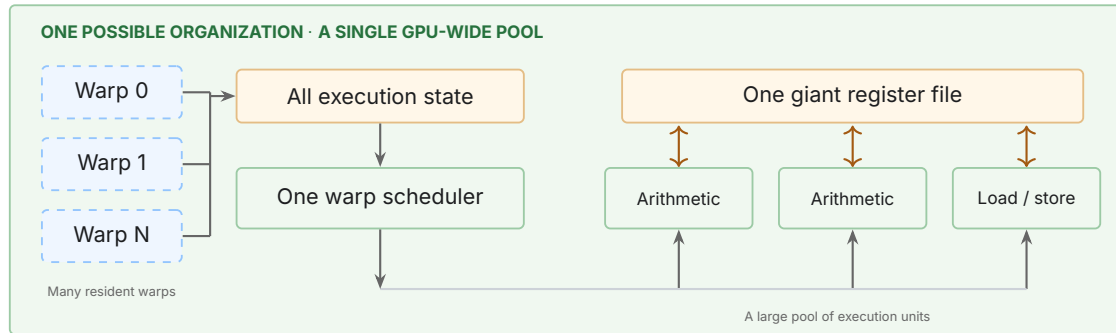
Track progress, choose an instruction, execute it

왼쪽 표는 하드웨어가 warp별로 추적하는 실행 상태의 예입니다. warp 0은 메모리에서 읽는 값을 기다리고, warp 1은 덧셈을 할 준비가 됐고, warp 2는 결과를 쓸 준비가 됐습니다. 레지스터에 담은 작업값과 함께 이런 진행 상황과 준비 여부도 기억해야 합니다.

가운데 warp 스케줄러는 이 상태를 참조해, 필요한 값과 실행 자원이 준비된 명령을 골라 발행합니다. 발행은 화살표에 적힌 선택한 명령을 실행 장치로 보내는 것입니다. 입력값을 기다리는 warp가 있으면 다른 warp의 준비된 명령을 고를 수 있습니다.

오른쪽 위의 산술 연산 유닛들은 덧셈과 곱셈을 하고, 아래의 load/store 유닛들은 메모리 읽기와 쓰기를 처리합니다. 초록 칸과 생략 기호는 여러 유닛을 나타내며, 개수는 예시입니다. 여기까지는 값 보관, 상태 추적, 명령 선택과 실행이라는 역할을 모았습니다. 이제 각 자원을 몇 개 두고 어떻게 연결할지 정해봅시다.

Why not build one giant execution engine?



One place for values. One place to choose work. Many units to execute it.

Why not build one giant execution engine?

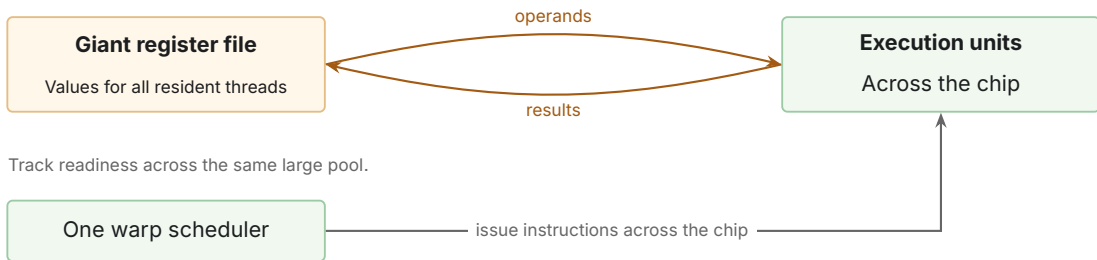
이제 필요한 역할을 실제 부품의 개수와 연결 방식으로 옮겨보겠습니다. 한 가지 안으로 GPU 전체를 하나의 큰 실행 엔진으로 만든다고 상상해봅시다. 많은 warp의 실행 상태를 가운데에서 모두 추적하고, 각 thread의 값은 오른쪽의 거대한 레지스터 파일 하나에 넣습니다. 스케줄러는 하나를 두어 그중 준비된 명령을 골라 아래의 많은 실행 유닛으로 보냅니다.

이렇게 실행 상태와 자원을 배정받아 현재 하드웨어에 머무는 warp를 resident warp라고 합니다. 제출한 모든 작업이 한꺼번에 올라온다는 뜻은 아닙니다. 지금은 올라온 warp들을 한곳에서 관리하는 설계를 생각하는 것입니다. 레지스터 파일 하나를 같이 사용해도 각 thread의 레지스터 값은 여전히 따로 배정됩니다.

얼핏 보면 충분해 보입니다. 일을 더 많이 하려면 레지스터 파일을 키우고, 더 많은 warp와 연산기를 넣으면 되니까요. 그런데 스케줄러가 멀리 떨어진 연산기를 골랐다면, 명령만 보내면 될까요?

The values and control signals must travel too

THE SAME CENTRALIZED DESIGN · NOW CONSIDER THE CONNECTIONS



More units also need more register bandwidth and a larger control network.

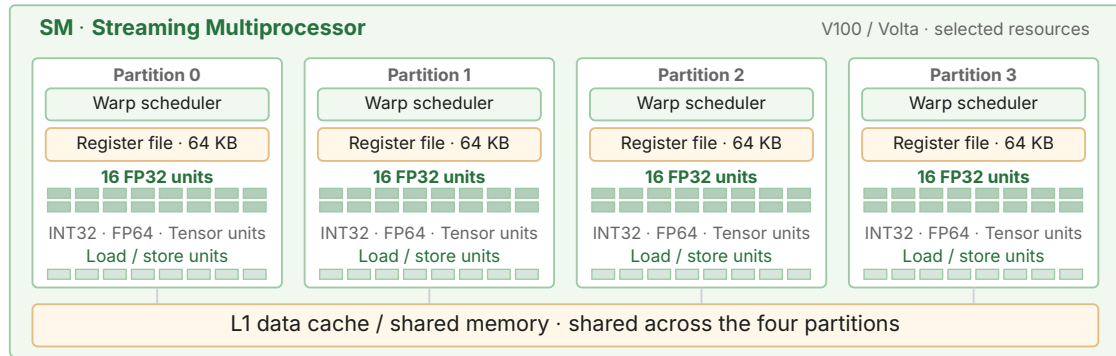
The values and control signals must travel too

스케줄러가 칩 반대편의 빈 연산기를 골랐다고 해봅시다. 명령만 도착한다고 계산할 수는 없습니다. 레지스터 파일에 있는 입력값도 그곳까지 가야 합니다. 계산이 끝나면 결과를 보관할 자리로 돌려보내 다음 명령이 사용할 수 있게 해야 합니다. 위쪽의 주황색 화살표가 그 왕복입니다.

연산기를 더 붙여 여러 명령을 처리하려면, 레지스터 파일도 그만큼 많은 값을 읽고 쓸 수 있어야 하고 그 값을 운반할 연결망도 필요합니다. 아래쪽 스케줄러도 넓은 범위의 준비 상태를 확인하고 여러 실행 유닛에 명령을 전달해야 합니다. 단순히 저장 용량과 연산기 개수만 늘려서는 끝나지 않는 것입니다.

이런 중앙 설계가 불가능하다는 뜻은 아닙니다. 커질수록 함께 해결해야 할 접근 대역폭과 연결 비용이 있다는 뜻입니다. 그렇다면 서로 자주 값을 주고받는 레지스터와 연산기, 이 실행들을 관리하는 장치를 가까이 묶고, 그 묶음을 여러 개 만들면 어떨까요?

An SM keeps execution and its working state close



Each scheduler manages many warps using its partition's local resources.

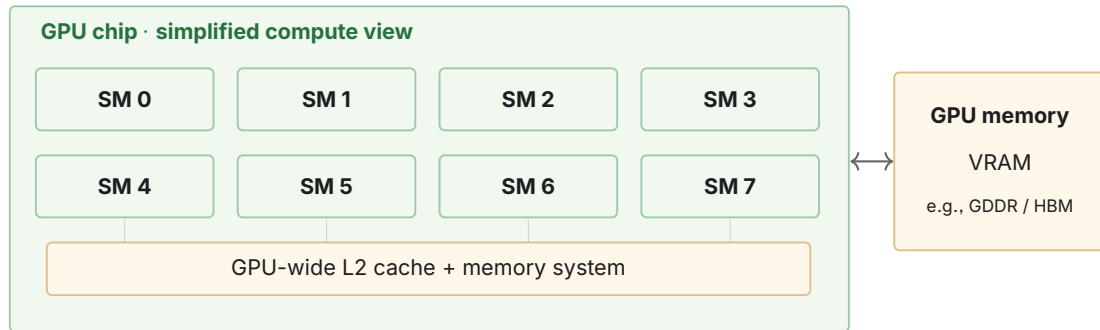
NVIDIA Volta whitepaper · Figure 5, pp. 12–13 · simplified and rearranged

An SM keeps execution and its working state close

앞에서 모은 자원들을 가까이 묶어 만든 실행 엔진이 SM, Streaming Multiprocessor입니다. 여기서는 V100의 Volta SM을 참고해 내부를 보겠습니다. 바깥 초록 테두리 전체가 SM 하나이고, 안에는 스케줄러와 레지스터 파일, 실행 유닛을 묶은 구획이 네 개 있습니다. 읽기 쉽게 네 열로 펼친 구성도입니다. 각 스케줄러는 자신이 담당하는 여러 warp의 준비된 명령을 골라 구획 안의 실행 자원에 공급합니다. 스케줄러 네 개가 warp 네 개를 뜻하지는 않습니다. 그 아래에는 thread들의 작업값을 담는 64 KB 레지스터 파일이 있고, 더 아래의 진한 초록 칸 16개는 FP32, 즉 32비트 부동소수점 연산 유닛입니다. 다른 종류의 산술 유닛들과 메모리 명령을 처리하는 load/store 유닛들도 같은 구획에 있습니다.

맨 아래 긴 L1데이터 캐시와 shared memory는 네 구획이 함께 쓰는 SM의 저장 자원입니다. 명령 캐시와 명령 전달 회로 같은 세부는 생략했습니다. 핵심은 실제 SM 안에서도 자주 주고받는 자원을 가까이 묶어 실행을 관리한다는 점입니다. 이제 이런 SM을 여러 개 배치해 GPU 전체로 확장해보겠습니다.

Scale the GPU by repeating that execution engine



Share control within a warp; keep execution resources local to an SM.

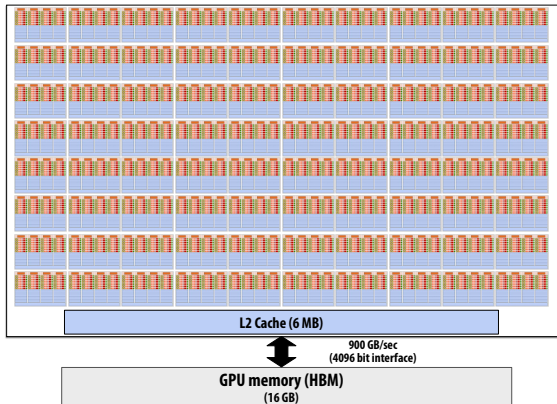
NVIDIA CUDA Programming Guide · GPU Hardware Model · illustrative SM count

Scale the GPU by repeating that execution engine

이제 자기 몫의 실행을 관리할 수 있는 SM을 여러 개 배치해 GPU를 키워봅시다. 각 SM은 가까운 레지스터와 실행 자원을 사용합니다. 그 바깥에는 여러 SM이 함께 사용하는 L2 캐시와 메모리 시스템이 연결되고, 오른쪽의 큰 GPU 메모리인 VRAM에는 텐서 같은 입력과 출력을 담습니다.

warp에서는 같은 명령의 제어를 함께 쓰고, SM마다 실행 상태와 연산 자원을 가까이 묶습니다. 그런 SM을 반복해서 배치하면 각각 자기 몫의 일을 진행할 수 있습니다. 실제 GPU의 전체 부품을 그린 것은 아니지만, 범용 계산을 담당하는 중심 구조를 이렇게 볼 수 있습니다. 이제 실제 칩의 예시와 연결해보겠습니다.

A concrete example: the NVIDIA V100



80 SMs

Repeated execution engines

Shared L2 cache

A cache shared by the SMs

HBM2 memory

Stores the inputs and outputs

Historical example, not a universal GPU specification.

Stanford CS149, Fall 2025 · slide 52 · V100 configuration shown in the lecture

A concrete example: the NVIDIA V100

[진행 메모: 왼쪽 그림 맨 위 줄의 맨 왼쪽 회색 직사각형 전체를 가리킨다. 내부의 네 세로 구획을 합친 묶음이 SM 하나다.]

Stanford 강의에 나온 NVIDIA V100의 사례입니다. 원본 자료의 색을 그대로 쓴 그림이라 앞의 도식과 색 의미가 다릅니다. 맨 위 줄의 맨 왼쪽을 보면, 주황색 막대 네 개와 그 아래 색 칸들을 함께 감싼 회색 직사각형이 있습니다. 이 묶음 전체가 SM 하나이고, 이 V100 구성에는 이런 SM이 80개 있습니다.

아래의 긴 상자는 여러 SM이 함께 사용하는 L2 캐시입니다. 그 바깥의 HBM2 메모리는 큰 입력과 출력을 저장합니다. 세대마다 개수와 구성은 달라져도, 여러 실행 엔진이 메모리 시스템에 연결된다는 구조를 확인할 수 있습니다. 그러면 이 여러 SM에 일을 나누는 것은 누구의 몫일까요?

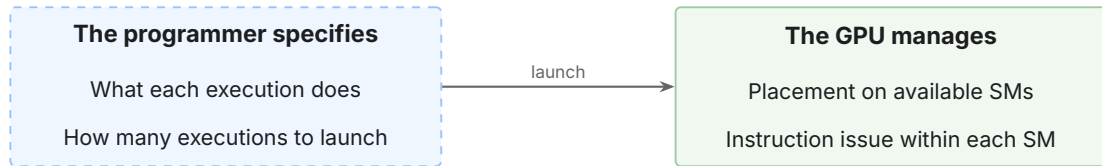
4. Following A + B

- From tensors to threads
- Scheduling and memory
- Utilization and completion

Following A + B

이제 A와 B를 더해 C에 담는 연산으로, 사용자와 GPU가 각각 무엇을 맡는지 살펴보겠습니다.

Describe the work; the GPU places it



PyTorch chooses CUDA implementations underneath tensor operations.

A work description is not a physical placement plan.

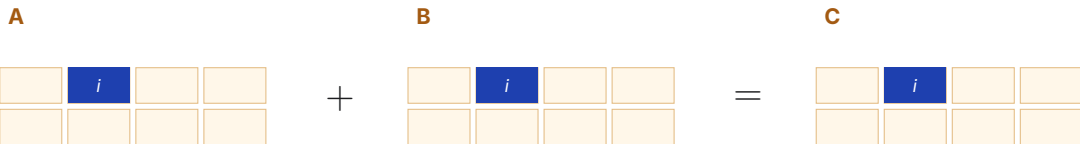
Describe the work; the GPU places it

이제 사용자의 입장에서 보면, CUDA 프로그래머의 몫은 할 일의 구성을 표현하는 것입니다. 각 실행이 무엇을 할지, 그 실행들을 어떻게 묶어서 얼마나 실행할지 정합니다. 어느 SM이 그 일을 맡을지는 GPU가 사용 가능한 자원에 맞춰 결정하고, 각 SM이 명령을 실행합니다.

PyTorch를 쓰면 커널을 고르고 실행 규모를 정하는 과정도 프레임워크가 맡습니다. 우리는 텐서에 대한 연산을 요청합니다. 다음 코드의 한 줄에서 시작해, 그 요청이 방금 본 SM과 warp에 어떻게 도착하는지 따라가보겠습니다.

Start with one line of PyTorch

```
import torch
A = torch.randn(1024, 1024, device="cuda")
B = torch.randn(1024, 1024, device="cuda")
C = A + B
```



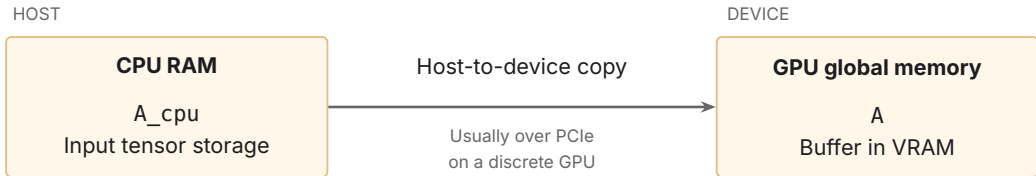
Each output needs only its own two inputs. No neighbor must finish first.

Start with one line of PyTorch

GPU에 있는 1024 곱하기 1024 행렬 두 개를 더하는 코드입니다. C는 A와 B의 같은 위치에 있는 원소끼리 더한 결과입니다. 행렬 전체를 하나의 숫자로 합치는 연산이 아니라, 같은 크기의 행렬을 만드는 원소별 덧셈입니다.

여기서 첫 번째 덧셈이 끝나야 두 번째 덧셈을 할 수 있을까요? 그렇지 않습니다. 각 결과에 필요한 것은 자기 위치의 입력 두 개뿐이고, 옆 원소의 결과를 기다릴 필요가 없습니다. 따라서 백만 개가 넘는 덧셈을 여러 실행에 나눠 맡길 수 있습니다. 처음에 보았던 많은 데이터에 같은 계산을 반복하는 구조입니다. 이 계산을 실행하기 전에, 먼저 입력이 어디에 준비되는지 확인하겠습니다.

If the input starts in CPU memory, copy it to the GPU



```
A_cpu = torch.randn(1024, 1024, device="cpu")  
A = A_cpu.to("cuda")
```

With `device="cuda"`, our original example already creates GPU tensors.

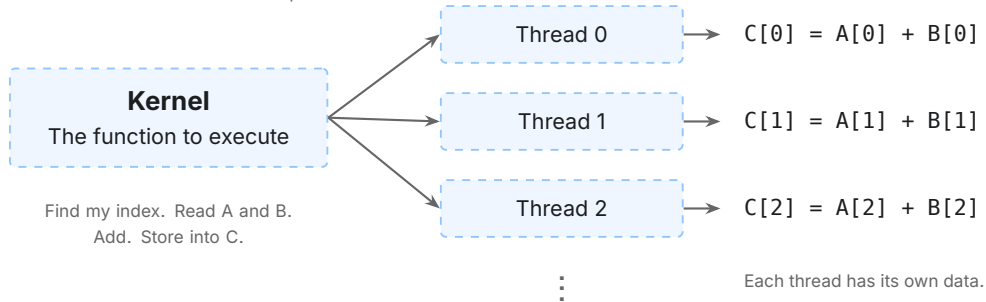
If the input starts in CPU memory, copy it to the GPU

원래 코드는 device에 cuda를 지정해 처음부터 GPU 텐서를 만듭니다. 여기서는 입력이 CPU에 있는 경우만 잠깐 보겠습니다. 파일에서 읽거나 CPU에서 만든 데이터는 처음에 왼쪽 RAM에 있습니다. 아래 첫 줄은 그런 출발점을 보여주고, 다음 줄의 `to cuda`는 그 값을 GPU 쪽 저장 공간으로 옮깁니다. 일반적인 별도 그래픽카드에서는 이 복사가 PCIe 연결을 지나고, 데이터는 VRAM에 있는 GPU의 global memory 버퍼에 놓입니다. B도 CPU에서 출발한다면 같은 방식으로 준비합니다.

이 입력 복사와, 커널이 실행되며 GPU 안에서 값을 읽는 동작은 별개입니다. 후자의 경로는 뒤에서 읽기 명령을 따라가며 보겠습니다. 어느 쪽에서 출발했든 이제 덧셈에 사용할 A와 B는 GPU에 준비되어 있습니다.

One program, many logical executions

Educational kernel · one element per thread · matrix elements numbered as a flat buffer



A kernel is code. A thread is one logical execution of that code.

CUDA Programming Guide · Kernel and thread model

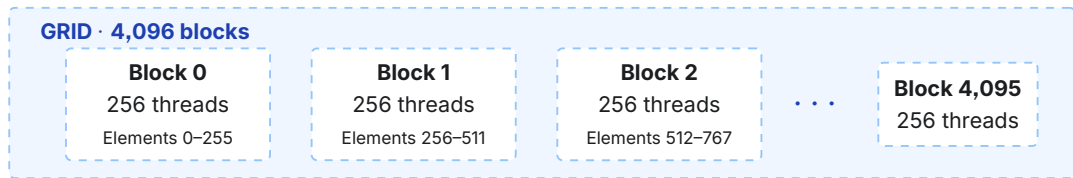
One program, many logical executions

이제 GPU에 준비된 입력으로 돌아와, 같은 덧셈을 thread 하나가 원소 하나를 맡는 교육용 커널로 표현하겠습니다. 실제 PyTorch는 한 thread에 여러 원소를 맡기는 등의 최적화를 사용할 수 있으므로, 여기서 고르는 실행 설정과는 다를 수 있습니다.

먼저 행렬의 원소를 행 순서대로 일렬로 번호 매겨, 1차원 버퍼처럼 보겠습니다. 이 그림에서 A의 0번은 첫 번째 행 전체가 아니라 맨 첫 원소이고, A의 1번은 그다음 원소입니다. GPU에서 실행할 함수가 커널이고, 그 함수를 실행하는 논리적인 인스턴스 하나가 thread입니다. 커널에는 자기 원소 번호를 알아내고, A와 B를 읽고, 더해서 C의 같은 위치에 저장하라는 내용을 씁니다. 왼쪽에는 프로그램이 하나만 있고, 오른쪽의 thread들은 각자 다른 원소 번호로 그 프로그램을 실행합니다. 같은 코드를 여러 번 실행하되, 각 실행이 서로 다른 데이터를 맡는 것입니다.

How many executions do we launch?

$1024 \times 1024 = \mathbf{1,048,576}$ elements



```
// CUDA C++: A/B/C buffers are ready; device_a/b/c are their GPU addresses.
add<<<4096, 256>>>(device_a, device_b, device_c, 1024 * 1024);
```

4,096 blocks $\times$ 256 threads = 1,048,576 logical threads.

CUDA Programming Guide · Thread blocks and grids

How many executions do we launch?

우리 행렬은 1024 곱하기 1024이므로 출력 원소가 104만 8576개입니다. 한 thread에 한 원소를 맡기면 논리적인 실행도 그만큼 필요합니다. CUDA는 thread를 block으로 묶고, 이번 커널 실행의 block 전체를 grid라고 부릅니다. 여기서는 block 하나에 thread 256개를 넣겠습니다. 이 크기는 우리가 고른 값입니다. 첫 block은 0번부터 255번까지, 다음 block은 256번부터 511번까지를 처리하므로 block은 4096개 필요합니다.

아래는 이 실행 요청을 표현한 CUDA C++ 코드입니다. 결과를 쓸 같은 크기의 GPU 버퍼 C도 미리 준비했다고 하겠습니다. device_a, device_b, device_c는 A, B, C 버퍼의 주소입니다. 꺾쇠 안의 두 숫자는 add 커널을 block 4096개로 실행하되 각 block에 thread 256개를 넣으라는 뜻입니다. 이렇게 커널과 실행 규모를 지정해서 제출하는 것이 kernel launch입니다. 여기까지 정한 것은 전체 작업의 구성이고, 실제 SM 배치는 GPU에 맡깁니다.

Every thread can find its own element

```
__global__ void add(
    const float* a,
    const float* b,
    float* c,
    int element_count
) {
    int index = blockIdx.x * blockDim.x
               + threadIdx.x;
    if (index < element_count) {
        c[index] = a[index] + b[index];
    }
}
```

Same code, different built-in IDs, different data.

BLOCK 3, THREAD 10

$$\underbrace{3 \times 256}_{\text{block offset}} + \underbrace{10}_{\text{local index}}$$

778

C[778] = A[778] + B[778]

The bounds check protects the final block when the size is not divisible by 256.

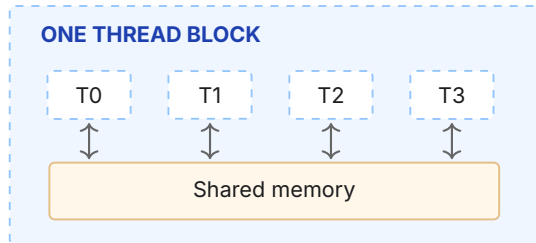
__global__: GPU kernel .x: 1D launch IDs

Every thread can find its own element

방금 호출한 add의 함수 내용입니다. 첫 줄의 global 표시는 GPU에서 실행할 커널이라는 뜻입니다. 소문자 a, b, c는 앞의 device_a, device_b, device_c 주소를 받습니다. 같은 코드에서 각자 맡은 원소를 구분할 때는 CUDA가 제공하는 번호를 사용합니다. blockIdx는 자기 block의 번호, blockDim은 한 block의 크기, threadIdx는 block 안에서 자기 번호입니다. 점 x는 이번에 사용하는 1차원 실행의 번호나 크기를 가리킵니다.

block 번호에 block 크기를 곱하면 그 block의 시작 원소 번호가 되고, 여기에 자기 thread 번호를 더합니다. 오른쪽의 block 3, thread 10은 3 곱하기 256에 10을 더한 778번입니다. 앞서 일렬로 붙인 원소 번호이므로, A와 B의 778번 원소를 읽어 C의 778번에 저장합니다. if문은 원소 수가 block 크기로 나누어떨어지지 않을 때 마지막 block의 불필요한 접근을 막습니다. 이렇게 각 thread가 자기 번호로 담당 데이터를 찾습니다.

A block is also a cooperation boundary



Share data locally

Threads in a block can exchange data.

Synchronize within the block

`__syncthreads()` provides a block-wide barrier.

For example, threads can combine intermediate results through shared memory.

This addition needs no shared intermediate values or block barrier.

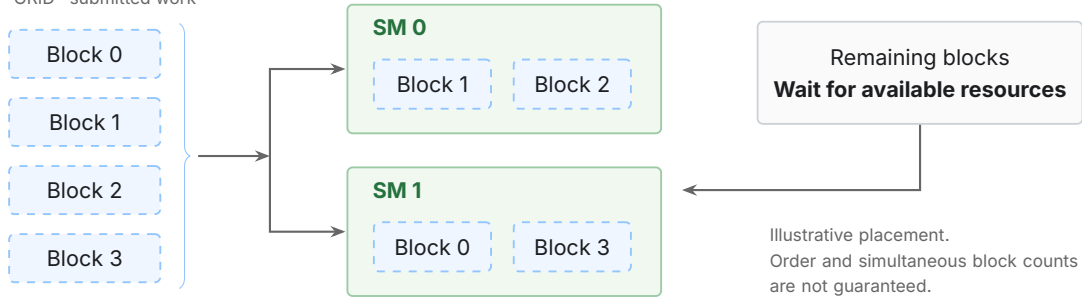
A block is also a cooperation boundary

block에는 실행을 묶는 것 외에도 역할이 있습니다. 같은 block의 thread들은 shared memory라는 가까운 저장 공간으로 데이터를 주고받을 수 있습니다. 또 `syncthreads`라는 동기화 지점에서 서로 기다렸다가 진행할 수 있습니다. 여러 thread가 각자 계산한 중간값을 모아 합치는 경우를 생각해볼 것입니다. 값을 shared memory에 써두고, 모두 쓸 때까지 기다린 뒤 다른 thread의 값도 읽는 식으로 협력할 수 있습니다.

이번 원소별 덧셈에는 자기 입력 두 개만 있으면 됩니다. 서로의 중간값을 읽거나 기다릴 일이 없으므로 이 협력 기능을 사용하지 않습니다. 이제 준비한 block들을 실제 실행 엔진에 배치해보겠습니다.

Many blocks share a finite set of SMs

GRID · submitted work



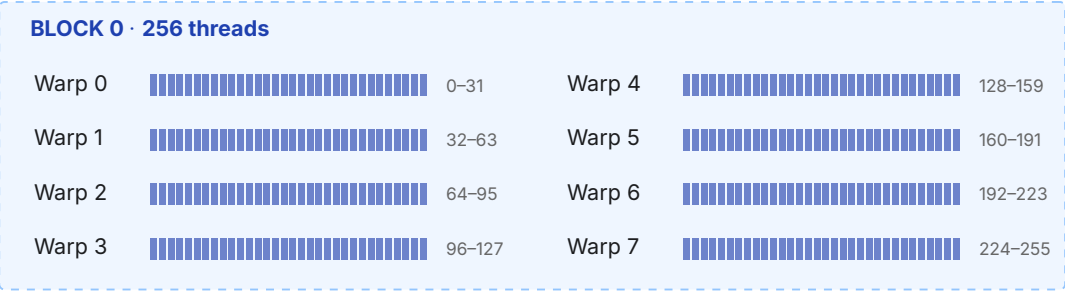
One block runs on one SM. An SM may host several blocks.

Many blocks share a finite set of SMs

GPU는 제출한 grid의 block들을 SM에 배치합니다. 왼쪽의 작업 묶음이 가운데 실행 엔진에 올라가는 모습입니다. 그림은 가능한 배치의 예이고, 실제 순서와 동시에 올라가는 block 수는 GPU와 커널에 따라 달라집니다.

중요한 규칙은 block 하나가 같은 SM에서 실행된다는 점입니다. SM 하나에는 레지스터, shared memory, 실행 상태를 보관할 공간이 허용하는 만큼 여러 block이 함께 올라갈 수 있습니다. 자원이 차면 오른쪽의 나머지 block은 기다립니다. 실행 중인 block이 끝나 자리가 나면 다음 block이 배치됩니다. 그래서 block 4096개를 더 적은 수의 SM이 나누어 처리할 수 있습니다. 실행할 작업을 충분히 제출해두면, 유한한 하드웨어가 자원을 되쓰며 진행하는 것입니다.

A 256-thread block contains eight warps



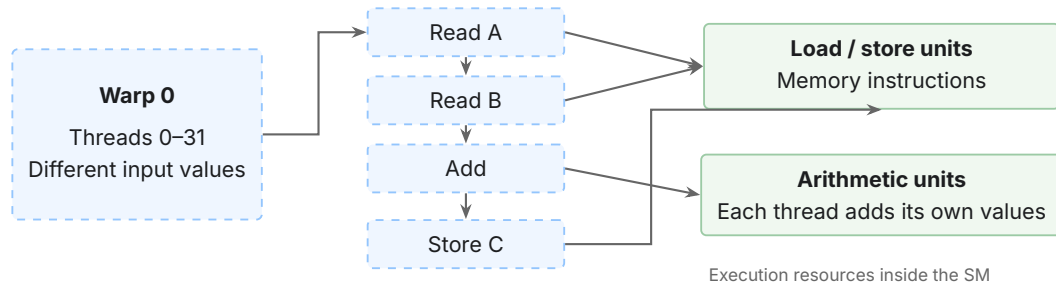
Within a block, consecutive groups of 32 threads form warps.

A 256-thread block contains eight warps

SM에 올라간 block 0을 확대해보겠습니다. 작은 파란 막대 하나가 thread 하나입니다. NVIDIA GPU는 같은 block 안에서 연속된 번호의 thread를 32개씩 묶어 warp로 관리합니다. 첫 warp는 thread 0번부터 31번, 다음 warp는 32번부터 63번을 말합니다. 이런 식으로 마지막 warp는 224번부터 255번을 말합니다.

우리는 256개 thread를 한 block으로 제출했고, SM은 이를 여덟 warp로 나누어 관리합니다. 이제 첫 번째 warp가 덧셈을 수행하는 과정을 가까이서 보겠습니다.

A warp needs different units for different instructions



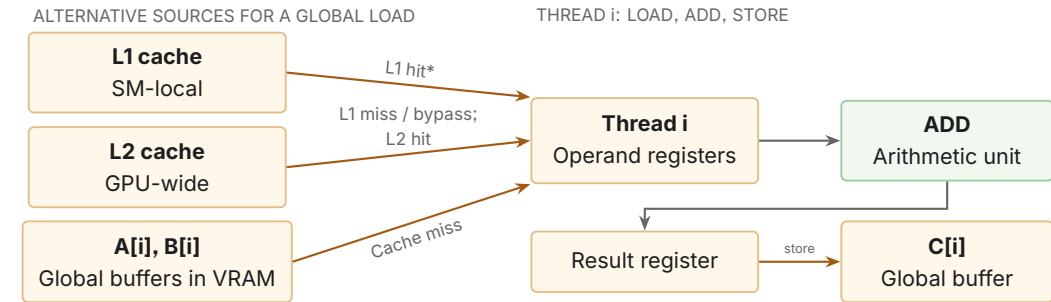
The instruction is shared; the data belongs to each thread.

A warp needs different units for different instructions

warp 0의 thread들이 해야 할 일은 같습니다. 자기 A 값을 읽고, 자기 B 값을 읽고, 둘을 더해 자기 C 위치에 저장합니다. 덧셈 명령이 발행되면 0번 thread는 A와 B의 0번 원소를, 31번 thread는 각 배열의 31번 원소를 더합니다. 명령은 함께 처리하고 데이터는 각자 맡는 SIMT가 이 계산에서도 나타납니다.

오른쪽을 보면 읽기와 쓰기는 load/store 유닛을 이용하고, 덧셈은 산술 연산 유닛을 사용합니다. 그림은 주소 계산 등의 세부 명령을 생략하고 네 종류의 작업에 집중했습니다. 각 명령은 그 종류에 맞는 SM의 자원을 사용합니다. 먼저 이 명령에 필요한 값이 어디에 보관되는지 보고, 이어 여러 warp의 명령이 번갈아 진행되는 모습을 보겠습니다.

For this addition, operands are loaded into thread registers



*L1 use depends on architecture and policy. Arrows show possible data sources, not physical routes.

The add kernel uses per-thread registers. Shared-memory staging is unnecessary.

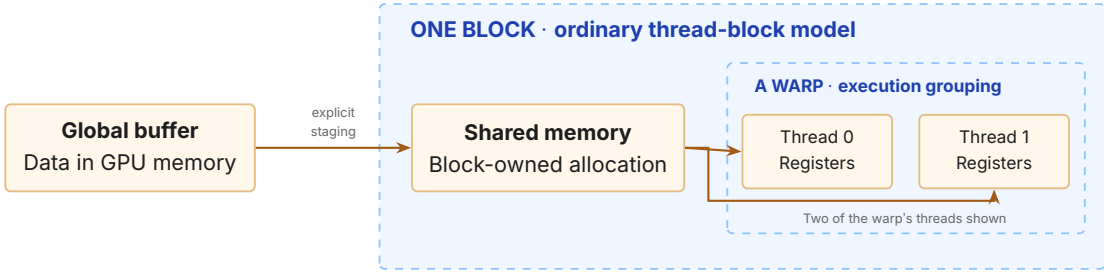
NVIDIA PTX ISA · Cache operators · logical data flow, not a physical floorplan

For this addition, operands are loaded into thread registers

GPU에 준비해 둔 A와 B의 값은 실제 ADD 명령까지 어떻게 도착할까요? thread는 자기 원소를 읽는 global load 명령으로 값을 가져와 작업용 레지스터에 둡니다. 왼쪽의 세 화살표는 이 읽기에 값을 줄 수 있는 공급원을 보여줍니다. L1을 사용하는 읽기에서 값이 L1에 있으면 거기서 얻습니다. L1에 없거나 L1을 사용하지 않는 읽기라면 L2가 처리할 수 있고, 캐시에도 값이 없으면 VRAM에서 가져와야 합니다. L1 사용 여부는 아키텍처와 명령의 정책에 따라 달라집니다. L2는 여러 SM이 공유하고, L1은 각 SM 안에 있는 캐시입니다.

어느 곳이 읽기를 처리하든 계산에 사용할 값은 오른쪽 thread의 레지스터에 놓입니다. 산술 연산 유닛이 두 값을 더하면 결과도 레지스터에 생기고, global store가 C의 해당 위치에 씁니다. 쓰기 경로는 별도의 캐시 정책을 따를 수 있습니다. 레지스터 값은 각 thread에 속하고, 물리적으로는 SM의 레지스터 파일에 보관됩니다. 이번 덧셈에는 이 값들을 shared memory로 따로 옮길 필요가 없습니다.

Shared memory is explicit storage for block cooperation



Useful when threads reuse or exchange data. This addition does not take this optional path.

Shared memory is optional staging that the kernel manages for cooperation and reuse.

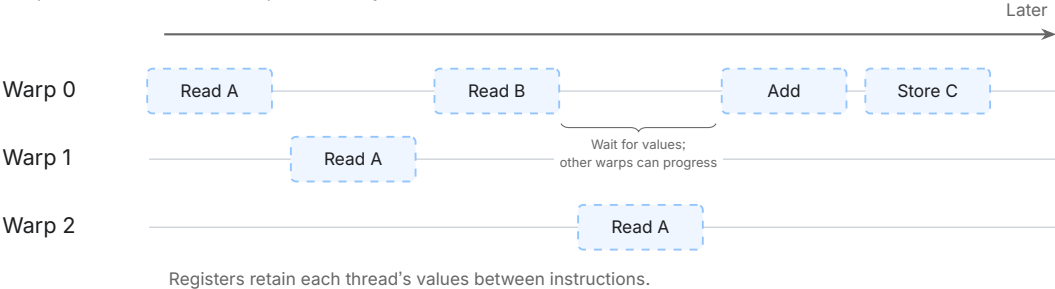
Shared memory is explicit storage for block cooperation

앞서 block 협력에 쓴다고 했던 shared memory를 이번에는 데이터 경로에서 보겠습니다. 캐시는 하드웨어가 관리하지만, shared memory에는 커널이 재사용하거나 주고받을 데이터를 명시적으로 준비합니다. 프로그램이 데이터를 쓰고 필요한 동기화를 거친 뒤, 각 thread가 자기 레지스터로 값을 읽어 계산합니다. 그림의 화살표도 두 thread의 레지스터에 닿습니다. 점선 warp 경계는 실행 묶음을 표시하며, 별도의 메모리 계층을 추가하지 않습니다.

L1과 shared memory가 같은 물리 저장 자원을 나누어 쓰는 GPU도 있지만, 프로그램에서 쓰는 방식은 다릅니다. shared memory는 협력과 재사용이 필요한 커널이 선택하는 저장 공간입니다. 이번 덧셈으로 돌아오면 앞 장의 레지스터 경로를 사용합니다. 이제 값이 오기를 기다리는 동안 SM이 다른 warp의 명령을 어떻게 진행시키는지 보겠습니다.

The scheduler issues the next ready instruction

One possible issue order · conceptual, not a cycle-accurate trace



A warp does not have to finish its whole program before another runs.

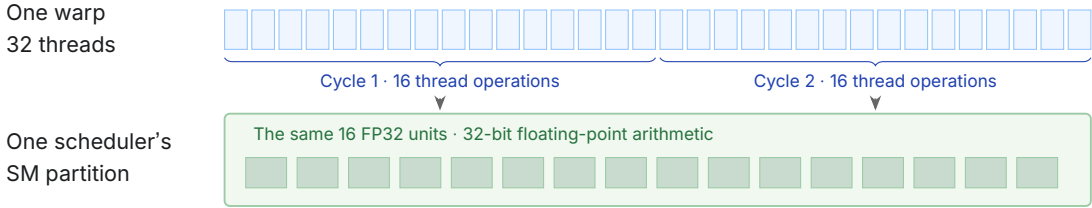
The scheduler issues the next ready instruction

스케줄러는 준비된 warp의 다음 명령을 골라 발행합니다. warp 하나를 끝낸 뒤에야 다른 warp를 시작하는 방식은 아닙니다. 그림은 가능한 발행 순서의 예입니다. warp 0이 A를 읽는 명령을 내보낸 뒤 warp 1도 A를 읽고, 이어 warp 0의 B 읽기와 warp 2의 A 읽기가 진행됩니다. warp 0이 값을 기다리는 동안에도 다른 warp의 준비된 명령을 발행할 수 있습니다. 값이 준비되면 warp 0은 더하고 C에 저장합니다.

그 사이 각 thread의 값은 레지스터에 보관됩니다. 실행 상태를 유지하면서 준비된 일을 고르는 것, 이것이 SM에 레지스터와 스케줄러를 연산기 가까이 둔 이유입니다.

Warp width and execution width are different numbers

VOLTA / V100 EXAMPLE



A 32-thread FP32 arithmetic instruction is processed over **two cycles**.

One of four V100 scheduler partitions · conceptual grouping · result latency is a separate quantity

One warp's 32-thread instruction uses its partition's execution width over time.

Warp width and execution width are different numbers

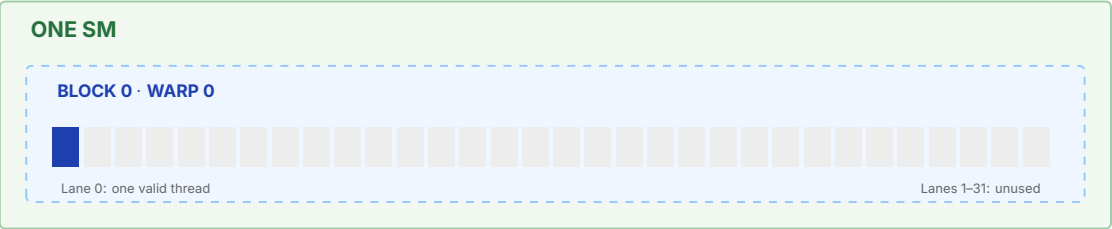
방금 발행한 warp의 명령 하나를 확대해보겠습니다. V100의 Volta SM에는 스케줄러 구획이 네 개 있습니다. 각 스케줄러가 맡는 warp 집합과 전용 산술 자원이 정해져 있으므로, 지금 보는 warp도 담당 스케줄러의 구획에서 처리됩니다.

이 한 구획에는 FP32, 즉 32비트 부동소수점 산술 연산을 하는 유닛이 16개 있습니다. 그림은 32개 thread의 산술 작업을 두 사이클에 나눠 같은 16개 유닛으로 처리하는 모습을 보여줍니다. 파란색은 처리할 일이고, 초록색은 두 차례 사용하는 같은 하드웨어입니다.

여기서 두 사이클은 32개 thread의 명령을 처리하는 폭에 관한 설명입니다. 명령의 결과가 준비되어 다음 계산에서 쓸 수 있을 때까지의 지연은 별도로 봐야 합니다. Volta의 이 사례에서는 warp의 크기가 32여도 실제 처리 폭은 16이고, 나머지는 시간에 나눠 처리합니다.

What if we launch just one thread?

```
add<<<1, 1>>>(device_a, device_b, device_c, 1);
```



One active thread in a warp on one SM.

What if we launch just one thread?

지금까지는 백만 개 넘는 원소를 GPU에 맡겼습니다. 이제 자원을 얼마나 활용할 수 있는지 보려고, 원소 하나만 더하도록 일을 줄여보겠습니다. GPU에는 다른 작업이 없다고 하겠습니다. 코드의 마지막 인자도 원소 수 1로 바뀌었고, block 하나에 thread 하나를 넣었습니다.

이때도 실행을 관리할 warp가 생깁니다. warp 안에서 0번부터 31번까지의 논리적인 thread 자리를 lane이라고 부릅니다. 실제 작업을 가진 thread는 lane 0 하나뿐이고 나머지 자리는 비어 있습니다. 이 block은 한 SM에 배치되고, 그 thread는 읽기와 덧셈, 쓰기에 필요한 자원을 차례로 사용합니다. 따라서 이 요청만으로는 warp의 나머지 자리와 다른 SM에 줄 일이 없습니다.

More threads in one block still means one SM

ONE LARGE BLOCK

```
add<<<1, 1024>>>(...)
```

Small example · 1,024 elements



1,024 threads/block: device and kernel limits apply

MANY BLOCKS

```
add<<<4096, 256>>>(...)
```

Original example · 1,048,576 elements



Blocks available for multiple SMs

Fill the warps, and provide blocks that can spread across SMs.

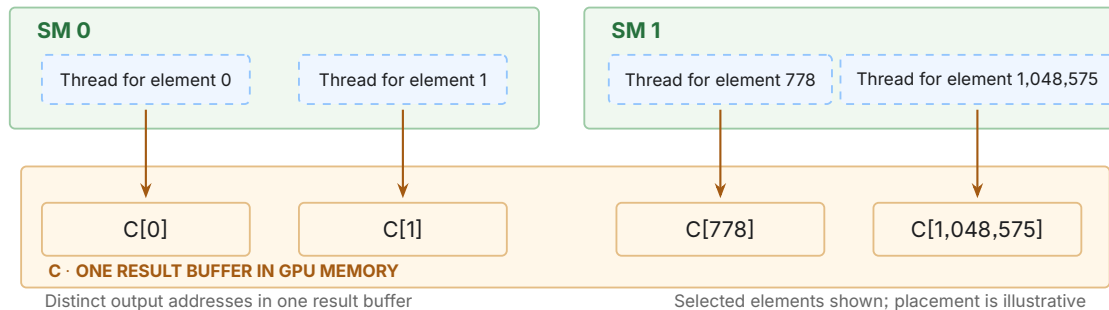
More threads in one block still means one SM

일을 늘리면 어떻게 달라질까요? 왼쪽은 원소 1024개를 처리하는 작은 예제이고, 오른쪽은 원래의 백만 개 넘는 원소를 처리하는 예제입니다. 작업량도 함께 바뀝니다. 여기서는 여러 SM에 일을 줄 수 있는 조건을 보겠습니다.

왼쪽은 block 하나에 thread 1024개를 넣어 warp 32개를 만듭니다. 1024는 NVIDIA GPU에서 대표적인 block당 thread 상한이며, 커널의 레지스터 사용량 같은 자원 제약도 충족해야 합니다. 허용되는 설정이어도 block 하나는 한 SM에서 실행됩니다. block을 키우는 것만으로 여러 SM에 일이 퍼지지는 않습니다.

오른쪽은 block 4096개, block당 thread 256개입니다. 여러 SM에 나눠 맡길 block이 충분히 있습니다. 그림의 SM 네 개는 배치 원리를 보여줍니다. GPU를 활용하려면 warp 안에 할 일이 있어야 하고, 여러 SM에 올릴 block도 있어야 합니다.

Threads write directly into C



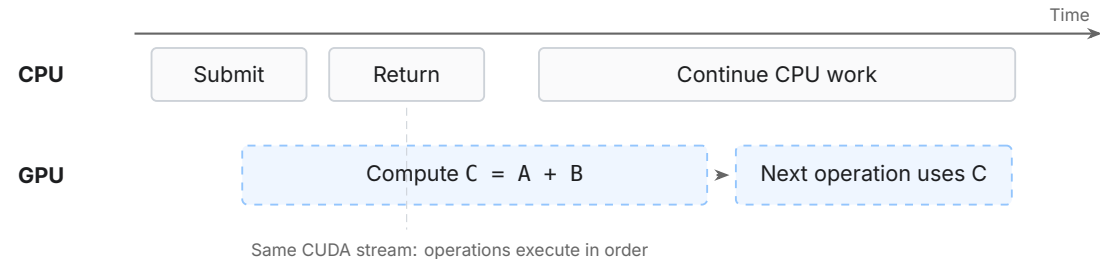
Each thread stores its own result. The CPU does not reassemble the matrix.

Threads write directly into C

이제 원래의 block 4096개 예제로 돌아와 결과가 어디에 저장되는지 보겠습니다. 각 thread는 자기 원소를 계산한 다음, launch 전에 준비해 둔 GPU 메모리의 C에 직접 씁니다. 그림의 주황색 공통 경계가 하나의 결과 버퍼이고, 그 안에서 서로 다른 출력 위치를 말합니다. 몇 개의 원소만 골라 그린 것입니다.

모든 thread의 저장이 끝나면 그 버퍼에 결과 행렬이 완성됩니다. 각 thread가 처음부터 최종 위치에 쓰므로 CPU가 SM별 조각을 받아 다시 조립할 필요도 없습니다. 그러면 Python에서 C를 받았을 때 이 값들은 모두 준비되어 있을까요?

Python can return before the GPU finishes



- GPU operations are normally submitted asynchronously.
- Ordinary CPU copies wait as needed; GPU timing must account for completion.

PyTorch documentation · CUDA semantics · illustrative timeline

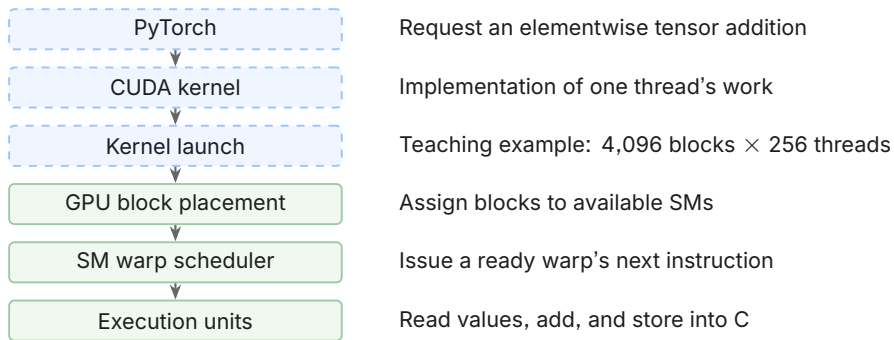
Python can return before the GPU finishes

Python은 결과를 담을 GPU 버퍼를 가리키는 텐서 객체 C를 받을 수 있지만, 그 버퍼의 값은 아직 계산 중일 수 있습니다. PyTorch의 GPU 연산은 기본적으로 비동기적으로 제출되기 때문입니다. 그림에서 CPU는 덧셈을 제출한 뒤 다음 일을 진행하고, GPU는 그 아래에서 계산을 계속합니다. C를 받는 시점과 값이 완성되는 시점을 나눠서 보면 됩니다.

CUDA stream은 GPU 작업을 순서대로 제출하는 통로입니다. 그림처럼 같은 stream에 C를 사용하는 다음 연산을 제출하면 앞선 덧셈 뒤에 실행되므로, 매번 CPU가 기다릴 필요는 없습니다. 서로 다른 stream 사이의 의존성은 별도로 관리합니다.

CPU로 값을 가져오는 일반적인 `C.cpu()` 복사는 PyTorch가 필요한 기다림을 수행합니다. 반면 GPU 계산 시간을 측정할 때는 완료 시점까지 포함하도록 동기화나 CUDA 이벤트를 사용해야 합니다. 호출 전후의 CPU 시간만으로는 GPU가 실제로 계산한 시간을 알 수 없습니다.

Follow the request all the way down



One tensor operation becomes many scheduled instructions on real hardware.

Follow the request all the way down

이제 한 줄의 요청과 실제 실행을 연결해보겠습니다. 이 그림은 위에서 아래로 작업의 표현, 구현, 하드웨어 실행의 관계를 보여줍니다. PyTorch에서는 A와 B를 더해 C에 담은 텐서 연산을 요청합니다. 그 연산을 GPU에서 수행하는 것이 CUDA 커널 구현입니다. 우리가 따라온 교육용 구현은 한 thread가 한 원소를 처리하며, block 4096개와 block당 thread 256개로 실행했습니다.

아래 초록색 부분에서는 GPU가 block을 자원이 있는 SM에 배치하고, SM의 스케줄러가 준비된 warp의 명령을 발행합니다. 입력은 GPU 버퍼에서 각 thread의 레지스터로 읽히고, 산술 유닛에서 더한 결과는 C의 담당 위치에 직접 기록됩니다. 이 저장까지 모두 끝나는 시점은 Python 호출의 반환 시점과 다를 수 있습니다. 한 줄의 요청이 여러 실행과 명령으로 나뉘어, 하나의 결과 버퍼를 완성하는 것입니다.

Keep the work and the machine separate

Kernel	The GPU program to execute
Thread	One logical execution of that program
Block	A group of threads that can cooperate
Grid	All blocks submitted for this kernel launch
Warp	A 32-lane execution group within one block
SM	The hardware engine that executes resident warps

Grid ≠ GPU. Block ≠ SM. Thread ≠ a dedicated core.

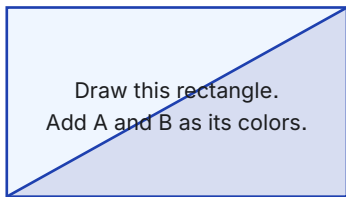
Keep the work and the machine separate

마지막으로 일과 기계의 관계 세 가지만 짚겠습니다. grid는 제출한 작업 전체이고, GPU는 그것을 처리하는 기계입니다. block은 한 SM에 배치되지만, 그 SM의 자원은 여러 block이 함께 쓸 수 있습니다. thread는 개별 실행이고, warp로 묶여 명령마다 필요한 하드웨어 자원을 사용합니다.

이 구별을 처음의 질문에 적용해보면, 데이터가 VRAM에 있다는 사실과 여러 SM이 계산하고 있다는 사실은 별개입니다. nvidia에서 메모리를 많이 사용한다고 해서 연산 자원도 그만큼 바쁘다고 볼 수는 없습니다. 또 프레임워크의 Python 호출이 돌아오는 것과 GPU 계산이 끝나는 것도 다릅니다. 이제 메모리에 둔 데이터, 제출한 작업, 실제 실행을 나누어 볼 수 있습니다.

From two triangles to one line

THEN



NOW

$$C = A + B$$

The constant idea: apply the same computation to many inputs.

From two triangles to one line

처음의 삼각형 두 개로 돌아왔습니다. 예전에는 이 사각형을 그려달라고 말하면서, 각 위치의 색은 A와 B를 더해서 정하라고 해야 했습니다. 지금은 프레임워크에서 A와 B를 더해 C에 담아달라고 요청하면 됩니다.

그 사이에 GPU가 활용하던 성질 자체가 사라진 것은 아닙니다. 많은 데이터에 같은 계산을 적용한다는 일은 계속 같았습니다. 달라진 것은 계산을 시키기 위해 그림을 그리는 척할 필요가 없어졌고, 그 일을 표현하고 실행하는 체계가 갖춰졌다는 점입니다. 오늘 본 thread와 block, warp와 SM은 그 한 줄과 실제 계산기를 연결하는 언어입니다. GPU를 다음에 사용하실 때 이 구조를 떠올릴 수 있으면 좋겠습니다. 감사합니다.

마지막으로 이 자료의 설명 범위를 말씀드리겠습니다. 오늘 본 커널과 다이어그램은 기본 개념을 이해하기 위한 교육용 예시입니다. 실제 하드웨어의 세부 구조와 메모리 동작, 실행 설정은 GPU 아키텍처와 소프트웨어 버전에 따라 달라질 수 있습니다. 특히 예제의 block 수와 thread 수를 실제 PyTorch 내부의 고정된 구현으로 받아들이지는 않으시면 좋겠습니다. NVIDIA와 Stanford의 자료는 출처를 표시하고 참고한 것이며, 이 발표에 대한 공식적인 보증이나 지지를 뜻하지 않습니다. 설명에 들어간 해석과 오류의 책임은 발표자에게 있습니다. 질문이나 수정 의견은 아래 연락처로 보내주시면 감사하겠습니다.

Educational scope This introduction uses simplified kernels and conceptual diagrams. Hardware details, memory behavior, and launch configurations vary by architecture and software version; the examples do not specify PyTorch's actual implementation. NVIDIA and Stanford materials are credited for reference and do not imply endorsement. Interpretations and any errors are the presenter's responsibility.